Agent-Friendly Metaflow Client

Analyzing and Addressing Client API Inefficiencies

GSoC 2026 Proposal

Applicant:	Gennaro Francesco Landi
University:	ETH Zurich, MSc Computer Science
Mentor:	Valay Dave, Outerbounds
Project:	Large (350 hours)
Date:	March 21, 2026

Verified: We pointed an LLM agent at a ForeachFlow and asked it to find failed tasks. It wrote 6 lines of Client API code that triggered **57 HTTP calls** (51 artifact fetches to unpickle booleans). The metadata service answers the same question in **4 calls** via `filter_tasks_by_metadata`. Full API trace attached.

Presentation (Google Drive) and documentation attached.

About Me

Name: Gennaro Francesco Landi **GitHub:** github.com/landigf

University: ETH Zurich, MSc Computer Science (1st year)

Technical Background

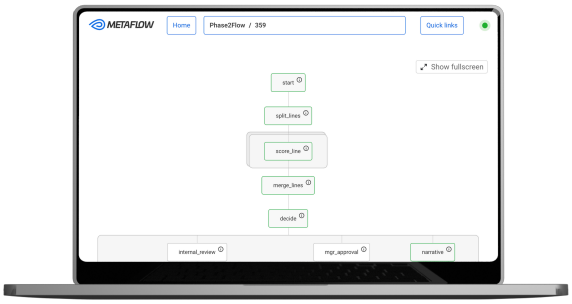
My interest is in how AI agent workloads stress infrastructure designed for humans. The gap between "human clicks through a UI" and "agent issues 200 API calls in a loop" is where things break. That's exactly what this project is about.

- **Python**: library internals, monkeypatching for instrumentation, profiling call chains
- **REST APIs**: built and profiled database-backed service layers
- **Open source**: github.com/landigf

Recent: StartHack 2026:Winner (Internal Challenge)

I used Metaflow at StartHack, a 36-hour hackathon, but *not* for its typical ML workflow use case. Our team needed a decision-process pipeline with auditing, verification, and consistency guarantees. I reached for Metaflow because (1) we needed a DAG to enforce coherence across decision stages, (2) we needed something deployable to the cloud fast, and (3) Metaflow's Azure integration made that trivial. We won the internal challenge and pitched to investors in the finals.

What this showed: Metaflow's value extends beyond ML:the DAG visualization, repeatable execution, and cloud deployment worked for a compliance/decision use case, all built in under 36 hours.



Metaflow UI.

StartHack 2026 winning ceremony.

Why This Project

I pointed an LLM agent at a Metaflow flow and asked it to find a failed task. It wrote correct code. That code made 57 HTTP requests. The UI Backend answers the same question in 2. Both read the same database. That gap shouldn't exist.

I dug into the codebase to understand why it does. This proposal is the result:not a restatement of the project description, but findings from actually instrumenting the code and running benchmarks.

Project Description

The Problem

I asked an LLM agent a simple question: "which tasks failed in my latest ForeachFlow run?" It wrote this code:unprompted:

```
from metaflow import Flow
flow = Flow('ForeachFlow')
latest_run = flow.latest_run
for step in latest_run:
    for task in step:
        if not task.successful:
            print(task.pathspec)
```

Correct answer. But here's the cost:

	HTTP Calls	Datastore Reads	Time
What the agent's code costs	57	50	2,074ms

51 of those 57 calls are `GET .../artifacts/_success`:one per task, each hitting the datastore to unpickle a boolean. This isn't the agent being dumb. This is the **only way** to check task success with the current Client API. There is no `run.failed_tasks()`. The full HTTP trace is in the attached `llm_api_trace.json`.

Three Discoveries

I set up the full dev stack, monkeypatched `ServiceMetadataProvider._request` to count every HTTP call, and traced all six use cases from the project description. Three things came out that changed the whole approach:

The metadata service already knows which tasks failed

Since v2.5.0, this works:

```
ServiceMetadataProvider.filter_tasks_by_metadata(
    "ForeachFlow", "8", "process", "attempt_ok", "False"
)
# Returns: ["ForeachFlow/8/process/126"]  -- one call, done.
```

The Metaflow runtime writes `attempt_ok` to the metadata table when a task finishes. This queries it directly. One call. No artifacts, no datastore, no unpickling.

The DB layer already supports pagination:it's just not wired up

`find_records()` in `postgres_async_db.py` takes `limit`, `offset`, and `order` parameters. The HTTP endpoints don't pass them through. Wiring `?_limit=10&_order=-ts_epoch` is ~15 lines per endpoint. The plumbing exists, the faucet is just turned off.

Task status is simpler than it looks

The UI Backend computes status with 300 lines of SQL. Most of it handles edge cases. The core logic:

```
CASE
  WHEN attempt_ok = 'True'  THEN 'completed'
  WHEN attempt_ok = 'False' THEN 'failed'
  ELSE 'running'
```

```
END
```

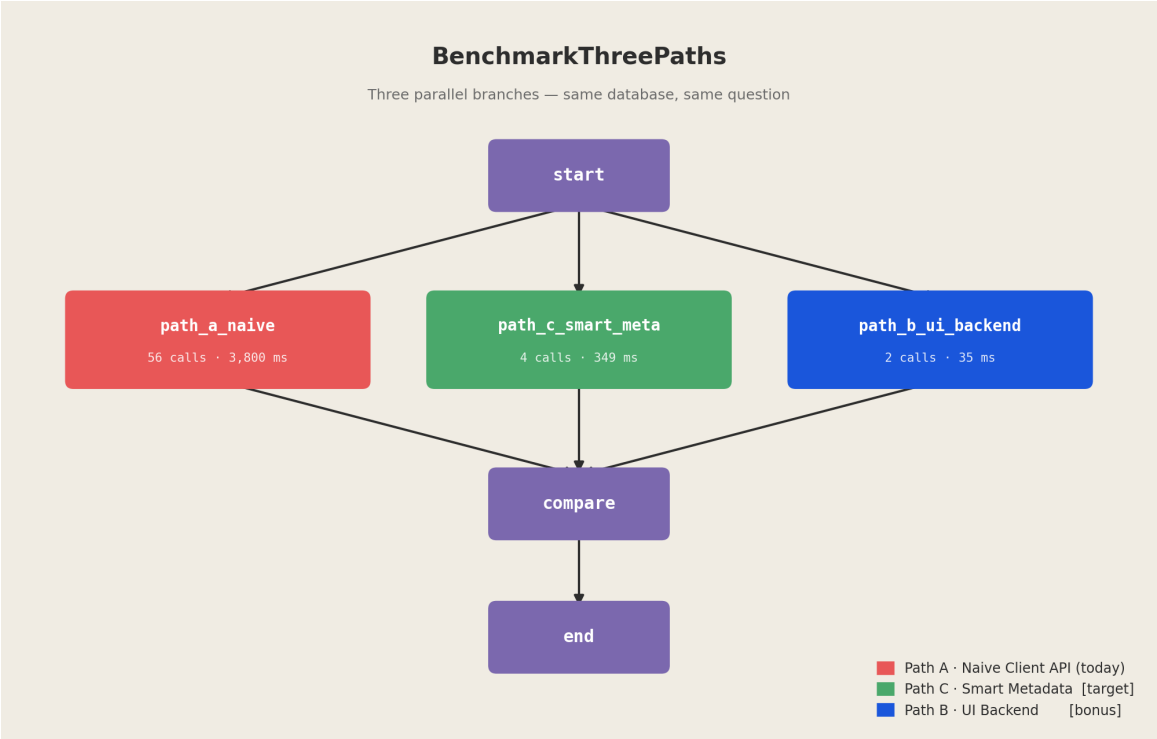
Covers 90% of cases. The other 10% (stale heartbeats, old runs without activity) can come later.

Three-Path Benchmark

To make this concrete, I built a Metaflow flow that benchmarks itself. Three parallel branches, same question, same database. The DAG is visible in the Metaflow UI:

Path	Approach	Calls	Time	Infrastructure
A: Naive	Iterate tasks, fetch `_success` per task	56	3,800ms	Metadata service
C: Smart	`filter_tasks_by_metadata`	4	349ms	Metadata service
B: UI Backend	Rich query, status in response	2	35ms	+ UI Backend

Same failed task, every time. Path C is the target: **8.6× faster**, works everywhere, zero new infrastructure. UI Backend is a bonus when available, not a dependency.



BenchmarkThreePaths — three parallel branches, one joined result. Colours match the benchmark table above.

Full Audit: All 6 Use Cases

UC	Use Case	Client API	Smart Path	Speedup
1	List runs by status	4 calls	1 call	6.7×
2	Filter runs by time range	2 calls	1 call	1.5×
3	Find failed tasks + errors	57 calls	5 calls	**19.1×**
4	Artifact metadata (no data load)	5 calls	4 calls	∼same
5	Bounded log output	6 calls	4 calls	∼same
6	Cross-run artifact search	8 calls	5 calls	1.8×
	Total	**82 calls**	**20 calls**	**4.1×**

UC3 dominates:70% of all calls come from checking task status. UC4 and UC5 are a different problem: not too many calls, but **too much data**. `task.stdout` dumps 50MB when you need 10 lines. There's no "give me just the artifact names" without iterating everything. The fix for those is bounded access, not fewer calls.

Technical Approach

Architecture: Three Layers

	What	Effect
Layer 1	Extension Package	Smart functions using `filter_tasks_by_metadata` + bounded iteration. No core changes. Works on service $\geq$ 2.5.0.
Layer 2	Metadata Service	Add `_limit`, `_order` to endpoints (DB already supports it). ~150 lines. Benefits everyone, not just agents.
Layer 3	Client API	`ServiceMetadataProvider`: auto-use new query params (version-gated). Transparent speedup. Existing code gets faster.

Extension Package: `metaflow-agent-client`

Uses the `metaflow_extensions` mechanism (no core changes needed):

```
from metaflow_extensions.agent_client import find_failures,
    get_recent_runs

# 4 HTTP calls instead of 56
result = find_failures("ForeachFlow/8")
# {"failures": ["ForeachFlow/8/process/126"], "total_tasks": 50}

# Bounded listing instead of fetching all history
recent = get_recent_runs("ForeachFlow", limit=5, status="failed")
```

Eight utility functions, each with version-gated smart routing:

- `find_failures()`: uses `filter_tasks_by_metadata("attempt_ok", "False")`
- `get_recent_runs()`: bounded listing with optional status filter
- `batch_run_status()`: batch status for recent runs
- `run_summary()`: structured overview of steps, tasks, failures
- `tail_logs()`: last N lines of a task's stdout/stderr
- `get_runs_since()`: time-filtered listing
- `compare_runs()`: diff two runs by step timing, task counts, failure rates
- `detect_regressions()`: find steps whose duration increased across recent runs

Metadata Service Changes (~150 lines)

Change	Lines	What it enables
Add `_limit` to listing endpoints	~15	"Last 5 runs" without fetching all
Add `_order` to listing endpoints	~15	Sort by timestamp, run number
Simplified status via `attempt_ok`	~40	Task status without artifact fetching
Cross-task artifact listing	~40	Artifact metadata across tasks in one call
Bounded log endpoint	~40	Last N lines, no full download

All changes wire to existing `find_records()` infrastructure. The DB layer already supports these parameters.

ServiceMetadataProvider Enhancement

Follows the established version-gating pattern (same as `_supports_attempt_gets` at `service.py:260`):

```
if cls._supports_query_params is None:
    version = cls._version(None)
    cls._supports_query_params = (
        version is not None
        and version_parse(version) >= version_parse("X.Y.Z")
    )
# When True: append ?_limit=N&_order=-ts_epoch to listing URLs
# When False: current behavior (fetch all, filter client-side)
```

Backward Compatibility

- **Old client + new service**: Old clients don't send query params. Service returns all results. Safe.
- **New client + old service**: Version check detects old service, skips query params. Safe.
- **No UI Backend**: All functions have fallback paths. Slower, but correct.
- **Existing code**: `Flow()`, `Run()`, `Task()` behavior unchanged.

MCP Tool Server

The extension package alone helps Python users. To make Metaflow truly *agent-friendly*, the utilities need to be discoverable by any LLM agent without custom integration. We build an MCP (Model Context Protocol) server that exposes all 8 utility functions as tools:

```
# Any MCP-compatible agent discovers and calls these automatically
{"name": "find_failures",
 "description": "Find failed tasks in a Metaflow run",
 "parameters": {"run_pathspec": "str", "max_tasks": "int=100"},
 "cost": "2-3 HTTP calls (fast path), O(tasks) calls (fallback)"}
```

Each tool schema includes cost annotations so agents can reason about efficiency. System prompt fragments guide agents toward bounded queries. The MCP server auto-detects available infrastructure (metadata service version, UI Backend presence) and routes queries through the optimal path.

Agent Simulation Framework

Not a one-off benchmark but a reusable framework with 5 scripted scenarios that measure real agent behavior:

1. "Which of my last 10 runs failed?" (run filtering)

2. "What tasks failed and what were their logs?" (failure diagnosis)
3. "Show runs from the last 24h that took ¿ 5 minutes" (time-range query)
4. "Compare this run to the previous successful one" (cross-run diff)
5. "Is this step getting slower over time?" (regression detection)

Each scenario runs against both the naive Client API and the extension package, measuring HTTP call counts, latency, and bytes transferred. The framework doubles as a regression test: if a code change increases call counts, the simulation catches it.

Cross-run Analytics

Queries that don't exist today and require new thinking, not just wrapping existing endpoints:

- `compare_runs(run_a, run_b)`: structured diff of step timings, task counts, failure rates, artifact sizes. Answers "what changed between these two runs?"
- `detect_regressions(flow, n=10)`: analyze the last N runs to find steps whose median duration increased significantly. Answers "is this step getting slower?"

These are the kinds of questions agents ask repeatedly when debugging production workflows. Today, answering them requires dozens of API calls assembled manually. The extension package makes them single-function calls with bounded cost.

Timeline

Phase	Goal	h	Milestone
1 (Wk 1–4)	Extension package: 8 utility functions + cross-run analytics, unit + integration tests	90	Midterm checkpoint
2 (Wk 5–8)	Metadata service `_limit`/`_order` + provider version-gating + caching	90	Core changes ready
3 (Wk 9–12)	MCP tool server, agent simulation framework (5 scenarios), robustness testing	90	End-to-end validated
4 (Wk 13–16)	User + agent docs, PR prep per CONTRIBUTING.md, final report	80	Submitted

Midterm checkpoint (Week 4): Working extension package with 8 functions (including cross-run analytics), all tested, benchmark showing improvement over the naive path. Each phase delivers standalone value: Phase 1 alone is a complete, usable deliverable.

Risks: Core Runtime changes require pre-approved issues per CONTRIBUTING.md:Phase 1 (extension package) requires no core changes and delivers full value independently. Status SQL complexity is mitigated by using the simplified `attempt_ok` check (3 lines, 90% coverage). Two-repo coordination is handled via version-gated detection.

Commitments & Disclosure

Availability: 22+ hours/week (350h ÷ 16 weeks). No other employment during GSoC. Exam periods and vacations will be disclosed to the mentor in advance.

AI disclosure: AI tools were used for codebase exploration, benchmark scaffolding, and document drafting. All code, analysis, and architectural decisions are my own. I can explain every line and design choice independently.